สหกิจศึกษา

ตำแหน่ง Backend Developer

ระบบ User Management System

ระยะเวลา: 3 ชั่วโมง	คะแนนเต็ม: 100 คะแนน	Node.js / Spring Boot	เกณฑ์ผ่าน: 70 คะแนน

หมายเหตุสำคัญ: โจทย์นี้ออกแบบให้ใกล้เคียงกับงานจริง นิสิตควรเขียน Code ที่ Production-ready โดยคำนึงถึง Security, Validation และ Maintainability ไม่ใช่แค่ทำให้ API ทำงานได้เท่านั้น

1. วัตถุประสงค์

แบบทดสอบนี้ประเมินความพร้อมปฏิบัติงานสหกิจศึกษาด้าน Backend Development โดยวัดทักษะ 3 ด้าน:

- ความรู้พื้นฐาน: RESTful API Design, HTTP Methods & Status Codes, Database Normalization
- ทักษะปฏิบัติ: Clean Code, Error Handling, Unit Testing
- Security Mindset: Broken Authentication, SQL Injection, Missing Authorization

2. ตัวเลือก Technology Stack

นิสิตเลือก Framework และ Database ได้ตามความถนัด แต่ต้องระบุในใบส่งงาน

Framework (เลือก 1):

Node.js + Express.js	Spring Boot
JavaScript / TypeScript	Java / Kotlin

Database (เลือก 1):

PostgreSQL	MySQL	MariaDB
แนะนำสำหรับ Production	นิยมใช้งานทั่วไป	MySQL-compatible

การเลือก Stack ไม่มีผลต่อคะแนน ผู้ประเมินจะพิจารณาที่คุณภาพ Code และครบถ้วนตามข้อกำหนด

3. โจทย์: ระบบจัดการผู้ใช้งาน (User Management System)

พัฒนา RESTful API ให้ครอบคลุมทุกหัวข้อด้านล่าง โดยใช้ Technology Stack ที่เลือกไว้

3.1 User CRUD — 15 คะแนน

Method	Endpoint	คำอธิบาย	Auth Required
GET	/api/users	ดึงรายการผู้ใช้ทั้งหมด (พร้อม Pagination)	Admin เท่านั้น
GET	/api/users/:id	ดึงข้อมูลผู้ใช้รายบุคคล	Admin หรือเจ้าของ Account
PUT	/api/users/:id	อัปเดตข้อมูลผู้ใช้	Admin หรือเจ้าของ Account
DELETE	/api/users/:id	ลบผู้ใช้	Admin เท่านั้น
PATCH	/api/users/:id/status	เปิด/ปิดการใช้งาน Account	Admin เท่านั้น

GET /api/users รองรับ: ?page=1&limit=10&search=keyword&role=user&sort=createdAt&order=desc

3.2 Authentication & Authorization — 20 คะแนน

ข้อนี้มีน้ำหนักคะแนนสูงสุด เนื่องจากเป็นทักษะที่จำเป็นที่สุดใน Backend จริง

Method	Endpoint	คำอธิบาย	Auth Required
POST	/api/auth/register	สมัครสมาชิก	Public
POST	/api/auth/login	เข้าสู่ระบบ — คืน JWT Access Token	Public
POST	/api/auth/logout	ออกจากระบบ (Blacklist Token)	ต้อง Login
GET	/api/auth/me	ดูข้อมูลตัวเอง	ต้อง Login
POST	/api/auth/refresh	ต่ออายุ Access Token	ต้องมี Refresh Token

ข้อกำหนด JWT:

- Access Token อายุ 15 นาที / Refresh Token อายุ 7 วัน
- เก็บ Refresh Token ในฐานข้อมูล เพื่อ Revoke ได้
- Blacklist Token เมื่อ Logout
- Middleware / Filter ตรวจสอบ Token และ Role ก่อนเข้า Protected Route ทุกเส้นทาง

Role-Based Access Control (RBAC):

- role: admin — จัดการผู้ใช้ทุกคนได้
- role: user — แก้ไขได้เฉพาะ Account ตัวเอง

3.3 Input Validation & Error Handling — 15 คะแนน

Validation Rules:

- name: ไม่ว่าง, ความยาว 2–100 ตัวอักษร
- email: รูปแบบถูกต้อง, ไม่ซ้ำในระบบ (เก็บเป็นตัวพิมพ์เล็กเสมอ)
- password: ขั้นต่ำ 8 ตัว, มีตัวเลขและตัวอักษรอย่างละอย่างน้อย 1 ตัว
- role: ต้องเป็น admin หรือ user เท่านั้น

รูปแบบ Error Response มาตรฐาน:

JSON — Error Response

```
{
  "success"   : false,
  "message"   : "Validation failed",
  "errors"    : [
    { "field": "email",    "message": "Invalid email format" },
    { "field": "password", "message": "Must be at least 8 characters" }
  ],
  "timestamp" : "2025-01-01T10:00:00.000Z"
}
```

HTTP Status Codes ที่ต้องใช้ให้ถูกต้อง:

- 2xx: 200 OK, 201 Created, 204 No Content
- 4xx: 400 Bad Request, 401 Unauthorized, 403 Forbidden, 404 Not Found, 409 Conflict
- 5xx: 500 Internal Server Error

> ต้องมี Global Error Handler (Express) หรือ @ControllerAdvice (Spring Boot) และไม่ส่ง Stack Trace กลับใน Production

3.4 Database Design & Migration — 10 คะแนน

ใช้ Database ที่เลือก + Docker Compose • Node.js: Sequelize/TypeORM/Prisma • Spring Boot: Spring Data JPA

Schema: ตาราง users

Field	Type	Constraint	หมายเหตุ
id	INT / BIGINT	PK, AUTO_INCREMENT	
name	VARCHAR(100)	NOT NULL	
email	VARCHAR(255)	UNIQUE, NOT NULL	เก็บเป็นตัวพิมพ์เล็กเสมอ
password	VARCHAR(255)	NOT NULL	เก็บเฉพาะ bcrypt hash
role	ENUM / VARCHAR	DEFAULT 'user'	ค่า: admin, user
is_active	BOOLEAN	DEFAULT TRUE	Soft Disable Account
refresh_token	VARCHAR(500)	NULLABLE	Refresh Token ล่าสุด
created_at	TIMESTAMP	DEFAULT NOW()	
updated_at	TIMESTAMP	ON UPDATE NOW()	

- ต้องมีไฟล์ Migration แยกต่างหาก (Flyway / Liquibase / Sequelize Migration / SQL script)
- ต้องมี Seed Data สร้าง Admin user เริ่มต้น พร้อม Credential ระบุใน README

3.5 Docker Compose — 10 คะแนน

ระบบทั้งหมดต้องรันได้ด้วยคำสั่งเดียว:

bash

```bash
docker compose up --build
```

Services ที่ต้องมีใน docker-compose.yml:

- app — Backend service (Node.js port 3000 / Spring Boot port 8080)
- db — PostgreSQL / MySQL / MariaDB พร้อม Volume ถาวร
- (Optional) adminer หรือ pgAdmin สำหรับ Database GUI

ข้อกำหนด:

- Health Check สำหรับ Database, app ต้องรอ db ก่อน (depends_on with condition)
- มีไฟล์ .env.example แสดง Environment Variables ทั้งหมด
- ห้าม Hardcode Password, Secret ใน Code หรือ docker-compose.yml

ตัวอย่าง Environment Variables (.env.example):

bash — .env.example

```bash
# App
PORT=3000                    # หรือ 8080 สำหรับ Spring Boot
NODE_ENV=development

# Database
DB_HOST=db
DB_PORT=5432                 # 5432=PostgreSQL  3306=MySQL/MariaDB
```

```
DB_NAME=userdb
DB_USER=appuser
DB_PASSWORD=changeme

# JWT
JWT_SECRET=your-super-secret-key
JWT_EXPIRES_IN=15m
JWT_REFRESH_EXPIRES_IN=7d
```

เกณฑ์ทดสอบ: ผู้ประเมินจะ Clone repo ใหม่ แล้วรัน docker compose up --build เพื่อดู API ที่ localhost:3000 (Node) หรือ localhost:8080 (Spring)

3.6 Security — 10 คะแนน

- bcrypt / BCryptPasswordEncoder: เข้ารหัส Password ด้วย salt rounds ≥ 10
- Environment Variables: DB credentials, JWT_SECRET ต้องอยู่ใน .env / application.yml
- ไม่ส่ง Password หรือ Sensitive Field กลับใน Response
- Rate Limiting: จำกัด /api/auth/login สูงสุด 5 ครั้ง/นาที
- Helmet.js (Node) / Security Headers (Spring): ตั้งค่า HTTP Security Headers
- (Bonus) CORS: กำหนด Allowed Origins ให้เหมาะสม

3.7 Pagination & Filtering — 5 คะแนน

GET /api/users ต้องคืน Response รูปแบบนี้:

JSON — Paginated Response

```
{
  "success" : true,
```

```
  "data"    : {
    "users" : [ { "id": 1, "name": "Alice", "email": "alice@example.com", ... } ],
    "pagination" : {
      "currentPage"  : 1,
      "totalPages"   : 5,
      "totalItems"   : 48,
      "itemsPerPage" : 10
    }
  }
}
```

Query Parameters ที่รองรับ:

- page, limit (default 10, max 100)
- search — ค้นหาจาก name หรือ email
- role — กรองตาม admin / user
- sort — createdAt / name / email | order — asc / desc

3.8 Unit / Integration Testing — 10 คะแนน

Node.js: Jest + Supertest • Spring Boot: JUnit 5 + MockMvc / RestAssured

ต้องเขียน Test ครอบคลุมอย่างน้อย 15 Test Cases:

Authentication Tests:

- Register: ข้อมูลถูกต้อง / email ซ้ำ / format ผิด
- Login: ถูกต้อง / password ผิด / email ไม่มีในระบบ
- Logout: ยืนยันว่า Token ถูก Revoke แล้ว

Authorization Tests:

- เรียก Protected Route โดยไม่มี Token ⟶ ต้องได้ 401

- User ธรรมดาพยายามลบ User อื่น ⟶ ต้องได้ 403
- Admin ลบ User ได้สำเร็จ

CRUD Tests:

- สร้าง User สำเร็จ / อัปเดตข้อมูลตัวเอง / Pagination ทำงานถูกต้อง

> Test coverage ต้องไม่ต่ำกว่า 60% • Node: npm test • Spring Boot: ./mvnw test หรือ ./gradlew test

3.9 API Documentation — 5 คะแนน

- Node.js: swagger-jsdoc + swagger-ui-express | Spring Boot: springdoc-openapi
- เอกสารครอบคลุมทุก Endpoint พร้อม Request / Response Example
- Node.js เข้าถึงที่ http://localhost:3000/api-docs | Spring Boot: http://localhost:8080/swagger-ui.html
- ระบุ Authentication Scheme (Bearer Token) ใน Swagger

4. โครงสร้าง Project ที่แนะนำ

Node.js + Express.js	Spring Boot
project/ ├── src/ │ ├── config/ │ ├── controllers/ │ ├── middlewares/ │ ├── models/ │ ├── routes/ │ ├── services/ │ └── utils/ ├── tests/ ├── migrations/	project/ └── src/main/java/ └── com/example/ ├── config/ ├── controller/ ├── dto/ ├── entity/ ├── exception/ ├── repository/ ├── security/ └── service/

```
├──── seeders/
├──── .env.example
├──── docker-compose.yml
├──── Dockerfile
└──── README.md
```

```
├──── src/test/java/
├──── src/main/resources/
│     └──── application.yml
├──── docker-compose.yml
├──── Dockerfile
└──── README.md
```

> โครงสร้างนี้เป็นคำแนะนำ ไม่ใช่ข้อบังคับ แต่การแบ่ง Layer ที่ดีจะได้รับพิจารณาในเกณฑ์ Code Quality

5. README.md ที่ต้องมี

1. ขั้นตอนติดตั้งและรัน Project ด้วย Docker Compose
2. Environment Variables ที่ต้องกำหนดและตัวอย่างค่า
3. วิธีรัน Database Migration และ Seed Data
4. วิธีรัน Test Suite (npm test / ./mvnw test)
5. Link ไปยัง API Documentation (Swagger)
6. Account Admin และ User สำหรับทดสอบที่สร้างจาก Seed
7. Framework และ Database ที่เลือกพร้อมเหตุผล

6. เกณฑ์การให้คะแนน

หัวข้อการประเมิน	คะแนนเต็ม	คะแนนที่ได้	หมายเหตุ
1. User CRUD API	15		
2. JWT Authentication & Authorization	20		
3. Input Validation & Error Handling	15		
4. Database Design & Migration	10		
5. Docker Compose	10		

หัวข้อการประเมิน	คะแนนเต็ม	คะแนนที่ได้	หมายเหตุ
6. Security (bcrypt, env, rate limit)	10		
7. Pagination & Filtering	5		
8. Unit / Integration Testing	10		
9. API Documentation	5		
รวม	**100**		

7. เกณฑ์ผ่าน

คะแนน	ระดับ	ความหมาย
85–100	ดีเยี่ยม (Excellent)	พร้อมปฏิบัติงานได้ทันที
70–84	ดี (Good)	พร้อมปฏิบัติงาน มีบางจุดที่ต้องพัฒนา
55–69	พอใช้ (Fair)	ต้องเสริมทักษะก่อนเริ่มงาน
< 55	ต้องปรับปรุง	ยังไม่พร้อมปฏิบัติงานสหกิจ

8. สิ่งที่ต้องส่ง

8. GitHub Repository URL (Public) — วาง Link ในตารางด้านล่าง
9. ตรวจสอบว่า Repo มีครบ: source code, Dockerfile, docker-compose.yml, .env.example, README.md, test files
10. ทดสอบก่อนส่ง: Clone repo ใหม่ ⟶ docker compose up --build ⟶ API ทำงาน ⟶ รัน test

ข้อห้าม: หลังส่ง GitHub URL แล้ว ห้ามแก้ไขข้อมูลใดๆ บน Repository นั้น การแก้ไขหลังส่งถือว่าผิดกติกา

9. เคล็ดลับสำหรับนิสิต

- เริ่มจาก Docker Compose และ Database ก่อน เพื่อให้ Infrastructure พร้อม
- ทำ Authentication ก่อน CRUD เพราะ CRUD ต้องอาศัย Auth Middleware / Security Filter

- เขียน Test ควบคู่กับ Code ไปพร้อมกัน อย่าทิ้งไว้ทำทีหลัง
- Commit บ่อยๆ พร้อม Commit Message ที่สื่อความหมาย (feat:, fix:, chore:)
- ใช้ .gitignore ให้ครอบคลุม: node_modules/, target/, .env, *.log
- บริหารเวลา 3 ชั่วโมงอย่างชาญฉลาด: วางโครงสร้าง 15 นาที ⟶ Core features 2 ชั่วโมง ⟶ Test + Docs 45 นาที

10. ส่งงาน

ส่ง Link GitHub Repository

— ขอให้โชคดีกับการทำแบบทดสอบ —